MindCheck

History-Aware Chess Plan Recognition
Using a Neural Network

Semester Project Proposal
AI2002: Artificial Intelligence | Fall 2026

National University of Computer and Emerging Sciences
School of Computing | Islamabad Campus

Selected Path: A - Software-Based

Group 29
Shahzad Ahmad | 24I-3090
Muhammad Ali | 24I-3158

Instructor: Dr. Muhammad Bilal

Requirements-aligned revision | 24 October 2026

Status: proposed work. No trained-model performance, dataset access, project reservation, or deployment is claimed as completed. Streamlit deployment and neural-model evaluation are mandatory deliverables. Dataset access and use permissions must be confirmed before the 25 October milestone.

1. Problem Statement and Real-World Justification

Beginner and intermediate chess learners can struggle to connect several opponent moves into a coherent strategic idea. MindCheck addresses this specific analysis task: given a game history, predict a plausible strategic plan and present the evidence in an understandable interface. The intended users are learners and students reviewing completed games or working through permitted training exercises.

The decision supported is which strategic threat or positional idea the learner should examine, and which response family deserves consideration. The system does not claim to reveal a player's true mental intention. It presents a learned hypothesis, supporting observations, and uncertainty. It is intended for post-game review and permitted practice, not unauthorized assistance during competitive games.

Expected impact: make selected strategic patterns easier to inspect during game review. Improved learning or playing strength will not be claimed without a separate user study. The semester evaluation focuses on classification quality, explanation support, and functional usability.

2. Selected Path and Objectives

This project selects exactly one category: Path A - Software-Based. It has no hardware component and is not submitted as a mixed-path project. Its core function is a trained neural network's classification of strategic plans, accessed through a working Streamlit application.

• Implement and train a history-aware neural classifier on encoded chess positions.

• Document a specific public PGN source, preprocessing pipeline, features, labels, splits, and limitations.

• Evaluate the classifier against rule and current-position baselines using held-out, independently reviewed examples.

• Deploy a complete Streamlit flow: game input, model execution, understandable output, and evidence inspection.

• Produce organized runnable code, a report with a proper workflow diagram, slides, evaluation evidence, and a live demonstration.

Primary research question: Does recent move history improve recognition of the selected strategic plans compared with using only the current board? Secondary question: Does adding symbolic evidence and temporal tracking improve useful prediction coverage or stability?

3. Scope and Priority

Priority	Components
Mandatory core	Public game data; five-plan taxonomy; training pipeline; trained residual CNN; independent evaluation; Streamlit input/inference/output flow; report and live demonstration.
Supporting functionality	Board facts, rule baseline, explicit neural-rule score fusion, simple temporal tracking, deterministic evidence-grounded explanations.
Optional after core completion	Stockfish-screened response recommendations; plan-conditioned threat simulation; LLM-assisted phrasing; extra plan classes.

Advanced search, CSP optimization, or an LLM are not prerequisites for the core classification task. They will not displace model training, ML evaluation, Streamlit deployment, or complete deliverables. A general chess-playing bot and reinforcement learning are outside scope.

4. Dataset Identification and Documentation

Selected proposed source: the Lichess Elite Database, a public collection of high-rated games in PGN form. The exact release/archive, provider page, available date range, filters, access conditions, and usage permissions must be recorded and confirmed by 25 October. This proposal does not assert that the archive has already been obtained or that redistribution is permitted. If access or permissions fail, any replacement requires documentation and instructor approval before changing the finalized dataset.

Initial processing target: enough games to obtain approximately 20,000-50,000 weakly labeled position windows, subject to available data, class coverage, and compute. These are planning estimates, not confirmed dataset counts. Preserve the acquired raw archive identifier and a reproducible selection manifest.

Dataset item	Planned treatment
Raw data	PGN move sequences and available game metadata; record exact fields after inspection.
Relevant features	Recent eight half-move board states, piece placement, side to move, castling rights, en-passant square, and history-validity mask.
Target	One dominant plan among five defined classes, or no dominant plan. Ambiguous examples may be excluded from weak-label training.
Preprocessing	Validate legal move sequences; reject malformed games; deduplicate; normalize orientation; construct history windows and padding; audit class counts.
Split	Game-level train/validation/test split, initially 70/15/15. Keep duplicate games together and check exact window overlap.
Limitations	Noisy automatic labels, overlapping strategies, class imbalance, opening repetition, and mismatch between high-rated games and novice games.

4.1 Weak training labels and independent test reference

Rule templates may inspect a bounded future segment to identify an executed pattern and create weak training labels. These labels describe retrospective evidence, not verified intention. Neural inputs contain only the current and preceding boards; future moves are never model features.

Create a reference set of approximately 200-300 held-out windows. Both members label them independently using a written rubric and only the history available at prediction time, without seeing model or automatic labels. Record confidence, supporting moves, and secondary plausible plans. Save initial judgments before resolving disagreements. Report raw agreement and Cohen's kappa, and disclose the reviewers' chess experience.

Student-reviewed labels are not expert ground truth. Seek an experienced-player audit if available; otherwise document this limitation. Keep retrospectively labeled and history-only evaluation tasks distinct. Do not evaluate the rule baseline solely against labels generated by those same rules.

5. Plan Taxonomy and Symbolic Evidence

Class	Operational evidence to define and test
Kingside pawn storm	Repeated flank-pawn advances toward the opposing king, with king-location and supporting-position context.
Queenside expansion	Queenside pawn advances associated with increased space or file access.

Class	Operational evidence to define and test
Outpost occupation	A weak target square and a piece maneuver toward or onto it.
Rook-file buildup	Rook access, alignment, or doubling on an open or semi-open file.
Pawn-chain undermining	Pawn moves that attack the base or support of a defined pawn chain.
No dominant plan	Insufficient or mixed evidence for the five selected plans; not a claim that no strategy exists.

Each template records preconditions, attacking side, evidence moves, progress indicators, and completion/invalidation conditions. The fact extractor uses python-chess board representations to compute selected predicates. Distinguish attacked squares from legal moves and restrict difficult concepts to definitions that can be unit-tested. Symbolic rules support evidence and comparison; they do not replace deployed neural inference.

6. Neural Algorithm and Use-Case Justification

Primary model: a compact residual convolutional neural network implemented and trained by the group in PyTorch. Start with 3-4 residual blocks and 64 channels, global average pooling, and a six-class classification head. Architecture size is a validation-controlled design choice, not a claim that these settings are optimal.

Why this model: board coordinates provide a consistent spatial representation, and stacking recent positions allows the classifier to use changes and piece maneuvers across time. The supervised output directly supports the intended task: choosing which plan hypothesis and related evidence to show during review. This suitability will be tested through a current-position-only baseline and a history-length ablation rather than assumed.

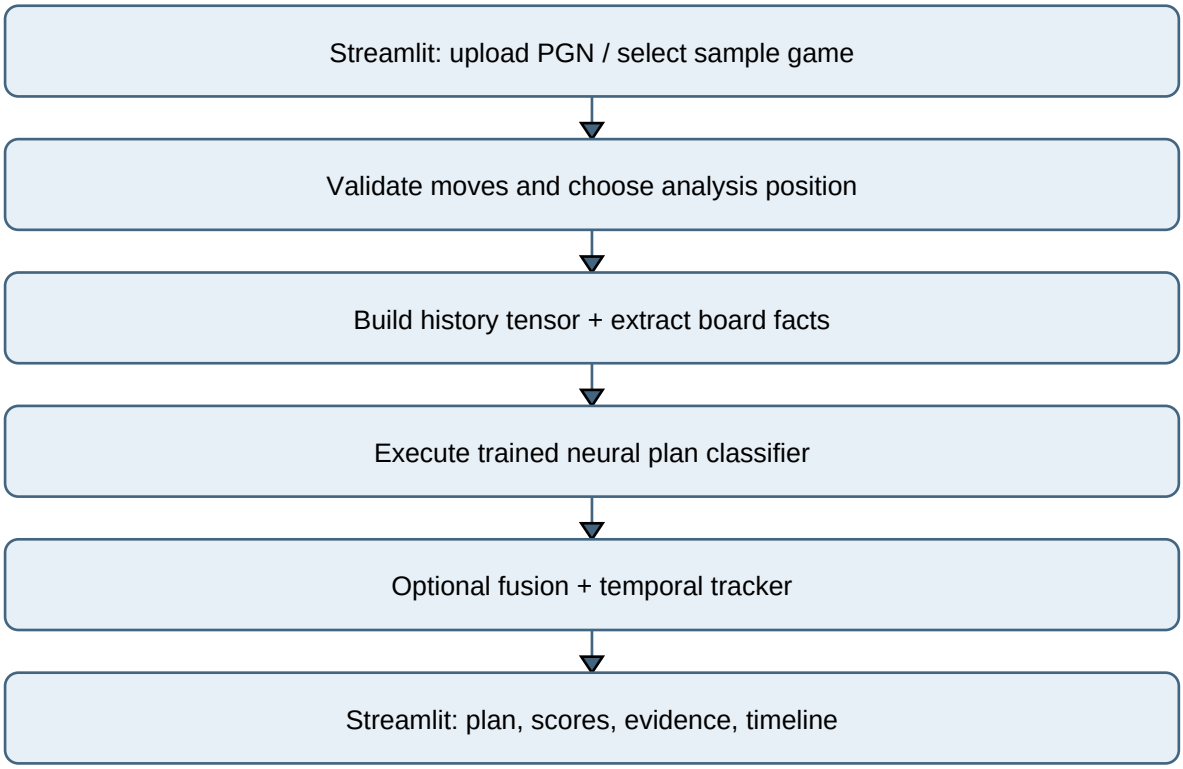
Input: a tensor of encoded recent 8 x 8 board states, oriented relative to the attacking side and accompanied by metadata planes and a padding-validity mask. Output: six softmax class scores. Softmax scores will be labeled model scores rather than calibrated certainty unless calibration is assessed.

Training: cross-entropy loss, Adam optimizer, validation-based early stopping, saved checkpoints, and reproducible seeds. Consider balanced sampling or class weighting if imbalance warrants it. Any label smoothing is selected on validation data. Record preprocessing version, hyperparameters, runtime, hardware, and class counts.

Primary comparison: current-position CNN versus history-aware CNN. Additional comparisons: rule-only detector and hybrid detector. Aim for three random seeds for main neural results if compute allows; report fewer runs transparently. Freeze the test set and all decision thresholds before final testing.

7. System Architecture and Pipeline Diagram

Figure 1. Required end-to-end software pipeline. All input, execution controls, predictions, and explanatory views are exposed through Streamlit.



Training: PGN -> weak labels -> game-level split -> CNN checkpoint

Training and evaluation are separate scripts/notebooks with reproducible commands. The deployed application loads the actual saved checkpoint and preprocessing configuration. If the checkpoint cannot load, the application reports model unavailability; it must not silently substitute rules and present those outputs as neural predictions.

7.1 Fusion and temporal tracking

For an optional hybrid output, blend normalized rule evidence with neural class scores using a validation-selected weight. Display neural-only and hybrid results separately in evaluation. Rule evidence is not automatically a calibrated probability. Abstain when the top score or top-two margin falls below validation-selected thresholds.

Maintain a plan state per attacking side. Initially require two supporting updates before activating a plan and two contradictory updates before switching. Estimate onset from recent supporting evidence; describe it as the earliest detected support, not the true beginning of intention. Display progress separately from danger. Evaluate tracking on an annotated subset if time permits.

8. Mandatory Streamlit Application and Deployment

The final submission includes a working Streamlit application, not only a notebook, command-line script, alternative frontend, or static screenshots. Streamlit is the primary demonstration interface. The team will confirm any hosting-specific expectations with the instructor; the project statement does not identify a mandatory hosting provider.

• Input: upload a supported PGN, select a sample game, and navigate to an analysis position.

• Processing: validate the game, construct the position history, load the checkpoint, and run inference when the user requests analysis.

• Output: show the chessboard, predicted plan or uncertainty, class scores, supporting moves/facts, and a short explanation.

• Usability: show model version, errors, missing-history conditions, and unavailable optional engine results clearly.

• Reproducibility: provide pinned dependencies, model artifact instructions, sample inputs, and a tested Streamlit launch command.

A single FEN is not sufficient to reconstruct move history. PGN is therefore the primary input. Any optional FEN mode must disclose its missing-history limitation and use the same documented padding policy as training. Cache results with history, model version, and preprocessing version, not FEN alone.

Deployment acceptance: the user can open the running Streamlit interface, enter/select a game, execute the trained model, and interpret the result without manually running intermediate scripts. Test a clean installation and the intended live-demo environment before submission.

9. Explanations and Optional Recommendations

Core explanations use controlled templates filled with checked board predicates and detector outputs. Distinguish observed facts from model hypotheses. For example: Recent moves suggest [plan]; supporting evidence includes [verified move pattern]. Do not present a score as proof of intention.

Optional LLM output must use a restricted structured claim schema with evidence IDs. Verify those claims and render approved text through templates; fall back to deterministic text on failure. No blanket zero-hallucination guarantee is made.

Optional Stockfish recommendations are legal moves screened using fixed, logged analysis settings. Report them as engine-assisted suggestions, not guaranteed safe counter-plans. If search-based threat estimation is included, define it consistently as the defender-perspective difference between proactive and ignore-plan rollout values under matched attacker policy and search budget. Mark unavailable estimates honestly and compare against a higher-budget reference. These features are stretch work only.

10. Testing, Evaluation, and Result Interpretation

Evaluation	Method	Evidence / metrics
Primary ML quality	Held-out independently reviewed windows; current-position vs history CNN	Macro-F1, per-class precision/recall/F1, confusion matrix, sample counts
Rule/hybrid comparison	Same fixed test examples and labels	Macro-F1, coverage, disagreement analysis
Uncertainty	Validation-chosen abstention threshold	Coverage and performance on accepted predictions; overall performance
History ablation	Compare one vs eight recent boards	F1 differences; matched training settings
Functional testing	Upload, malformed PGN, early game, unsupported/ambiguous plan, model-load failure	Test cases, expected/actual behavior, pass/fail logs
Runtime and demo	Complete Streamlit flow on stated hardware	Median/p95 inference or end-to-end time; failures
Optional explanation audit	Review accepted and rejected claims independently	Unsupported claims, retained useful claims, fallback rate

Use game-cluster bootstrap confidence intervals when windows share games. Report class balance, label disagreement, and representative false positives/negatives. Do not tune on the test set. Explain what measured scores imply within the five-plan scope, and do not generalize them to all chess strategy or learning outcomes.

Results are not available at proposal time. The final report must contain actual measured quantitative results and qualitative examples, including limitations and negative findings. Low model accuracy is not hidden by showcasing only rule or engine output; analyze the failure and demonstrate the real model honestly.

11. Tools, External Resources, and Original Work

Resource	Role and justification
Python / PyTorch	Group-implemented encoding, training, classifier, and evaluation.
python-chess	Board state, legal move validation, PGN parsing, and optional UCI integration; not the plan classifier.
Streamlit	Mandatory interactive application and demonstration interface.
Public PGN data	Training and evaluation source; record source terms and provenance.
Stockfish (optional)	Independent engine-assisted analysis and response screening; document settings and version.
LLM API/local model (optional)	Restricted wording assistance only; not the core learned solution.

No existing complete project, tutorial solution, or repository will be copied and submitted. The group will implement the task-specific data pipeline, labels, model integration, evaluation, and Streamlit workflow, document all reused resources, and explain each component during the demonstration. External libraries and APIs are support resources rather than substitutes for the group's implementation.

12. Deadline-Aligned Schedule

Dates / deadline	Work and milestone
11 Oct 2026	Course deadline for group/path finalization has passed. Confirm existing registration and Path A selection; no completion is assumed here.
24-25 Oct	Confirm unique reserved idea; finalize dataset archive/access, use case, primary CNN and scope. Submit required dataset/model/use-case form by 25 Oct.
26 Oct-1 Nov	Fact extractor, five-plan rubric, game validation and splits, Streamlit skeleton.
2-8 Nov	Weak-label pipeline, rule baseline, first CNN checkpoint.
9-15 Nov	Independent annotations, model comparison, Streamlit checkpoint inference.
16-22 Nov	Temporal tracking, evidence presentation, optional response screening; feature freeze at end of week.
23-29 Nov	Final evaluation, failure analysis, runtime testing, clean-install and demo checks.
30 Nov-6 Dec	Complete report, slides, code organization, artifacts and ZIP. Final submission on GCR by 6 Dec 2026.

Dates / deadline	Work and milestone
7-9 Dec	Live Streamlit presentation/demonstration; both members explain implementation.

The group must check that the idea is unique and correctly reserved in the allocation sheet. If the registered title or path differs from this proposal, obtain instructor/TA confirmation before implementation changes. The proposal itself does not establish reservation or approval.

13. Work Distribution

Component	Primary owner	Shared responsibility
Data pipeline and rules	Shahzad Ahmad	Plan rubric and evidence definitions
CNN training and evaluation	Shahzad Ahmad	Architecture review and error analysis
Streamlit application	Muhammad Ali	Interface design and acceptance testing
Temporal tracker and explanation	Muhammad Ali	Verification rules and integration
Optional engine analysis	Muhammad Ali	Budget and correctness review
Independent annotation	Both independently	Adjudication after initial labels
Report, slides, live demo	Both	Equal understanding of all components

14. Risk Management

• Dataset access or licensing uncertainty: verify by the finalization deadline and seek approval for any replacement.

• Noisy/ambiguous labels: independently review test labels, permit abstention, disclose limitations.

• Class imbalance or low accuracy: audit samples, adjust training using validation data, and retain honest baseline comparisons.

• Compute shortages: reduce channels, epochs or sample count rather than remove the trained neural component.

• Scope creep: prioritize model, evaluation and Streamlit; cut optional search and LLM features first.

• Deployment failures: test a fresh environment, store required model artifacts, and prepare local Streamlit execution and sample PGNs for the live demo.

• Academic integrity: maintain source/resource records, original task-specific code, and equal member understanding.

15. Mandatory Deliverables and Submission

• Source code: complete organized runnable project, meaningful names/comments, dependency file, sample inputs, and execution instructions.

• Project report: problem, real-world justification, selected path, objectives, methodology, algorithm, visual pipeline, dataset, implementation, testing, measured results, limitations, and conclusion.

• Live demonstration: working Streamlit application showing input, trained-model execution, and understandable predictions.

• Presentation slides: motivation, solution, model choice, implementation, qualitative and quantitative testing/results, limitations, and conclusion.

• Evaluation evidence: held-out metrics, confusion matrix, baseline comparisons, representative failures, and relevant test logs.

• Supporting artifacts: trained checkpoint, preprocessing configuration, label rubric, dataset manifest, and permitted annotations/data or retrieval instructions.

Package all required files in one ZIP and submit once per group on GCR by 6 December. Ensure the artifact is runnable from the included instructions. A hardware video is not a Path A mandatory deliverable in the supplied statement; a recording, if prepared voluntarily, does not replace the required live demonstration.

16. Compliance and Acceptance Checklist

Requirement	Completion evidence to provide
Exactly Path A	Explicit declaration in report and slides; no mixed-path submission.
Meaningful implemented ML	Saved trained CNN, inference code, and live model-generated classification.
Documented dataset	Exact source/release, terms, features, targets, preprocessing, splits, limitations.
Evaluated ML pipeline	Measured metrics, interpretation, baseline comparison and errors.
Working Streamlit deployment	Usable running interface with input -> model execution -> understandable output.
Proper pipeline diagram	Figure in this proposal and final report, updated to match actual implementation.
Representative testing	Functional test records and diverse held-out examples.
Complete submission	Single GCR ZIP, report, slides, code, evaluation evidence and live demo.
Originality and group understanding	Documented reuse, original task-specific implementation and both members able to explain it.

Success means a complete, reproducible learned classification system is actually demonstrated through Streamlit and evaluated honestly. Positive results, a particular F1 threshold, and superiority over rules are research outcomes to measure, not promises. Optional sophistication does not compensate for a missing mandatory requirement.

17. Basis and Reference Documentation

Primary requirements source: the user-supplied AI2002 Artificial Intelligence Semester Project Statement, Fall 2026, School of Computing, Islamabad Campus. The project scope is adapted from Group 29's supplied MindCheck proposal.

Implementation documentation to consult and cite in the final report: PyTorch documentation; Streamlit documentation; python-chess documentation; the exact selected Lichess Elite archive/provider documentation; and Stockfish documentation if used. Verify current source details and permissions before finalizing bibliographic entries. No external product novelty or numerical performance claims are needed for this proposal.